03_session_fundamentals

Video Script: Session Management Fundamentals

Duration: 10 minutes **Difficulty**: Beginner **Module**: 2 - Session Management **Goal**: Master session creation, modes, and lifecycle

---

[0:00-0:30] Introduction

[ON SCREEN: “Session Management Fundamentals”]

SCRIPT: “Welcome to Session Management! In this video, you’ll learn how to organize your AI-assisted development work into focused sessions. Think of sessions as workspaces - each one has a specific purpose, mode, and lifecycle. By the end of this video, you’ll be creating and managing sessions like a pro.”

[Visual: Animated workspace icons organizing into session containers]

[0:30-2:30] What Are Sessions?

[ON SCREEN: Session concept diagram]

SCRIPT: “A session in HelixCode is a focused development period with a specific goal. It’s like opening a new project in your IDE, but with AI context awareness.

Every session has: - A unique **name** that describes what you’re working on - A **mode** that optimizes the AI for that type of work - A **status** that tracks where you are in the lifecycle - A **project** association to keep things organized - **Tags** and **metadata** for easy filtering and search

Why use sessions? Imagine you’re working on authentication. You create an ‘implement-auth’ session, work for a few hours, get interrupted, come back tomorrow - and everything is exactly where you left it. Your conversation history, context, templates, everything.”

[Demo: Show session creation and restoration]

[2:30-5:00] The Six Session Modes

[ON SCREEN: Grid showing all 6 modes with icons]

SCRIPT: “HelixCode has six session modes, each optimized for a different type of work:

Planning Mode - This is where you design and architect. The AI focuses on high-level design, system architecture, and breaking down requirements into tasks. Use this when starting a new feature or refactoring.

Building Mode - The most common mode for implementation. The AI helps you write code, suggesting functions, handling boilerplate, and generating implementations. This is your main coding mode.

Testing Mode - All about quality assurance. The AI helps write tests, identify edge cases, suggest test scenarios, and even debug failing tests. Use this to ensure your code is bulletproof.

Refactoring Mode - Improving existing code. The AI identifies code smells, suggests optimizations, helps with renaming, and restructures code for better maintainability.

Debugging Mode - When things break. The AI analyzes errors, suggests root causes, helps trace issues, and proposes fixes. It’s like having a debugging partner.

Deployment Mode - Getting code to production. The AI helps with CI/CD, deployment scripts, configuration, monitoring setup, and release documentation.

Each mode changes how the AI thinks and responds, optimizing for that specific type of work.”

[Visual: Show example AI responses in different modes for the same question]

[5:00-7:30] Session Lifecycle

[ON SCREEN: Lifecycle state diagram]

SCRIPT: “Sessions have a lifecycle with five states. Let me show you with code:

Idle - Created but not started yet”

[Show code]:

```
// Create a session - starts as Idle
sess := sessionMgr.Create(
    "implement-payment-api",    // Name
    session.ModeBuilding,        // Mode
    "payment-service",           // Project
)

// Session is now in Idle status
fmt.Println(sess.Status) // "idle"
```

SCRIPT CONTINUES: "**Active** - You're working on it"

[Show code]:

```
// Start the session
err := sessionMgr.Start(sess.ID)

// Status changes: Idle → Active
fmt.Println(sess.Status) // "active"
fmt.Println(sess.StartedAt) // Current time
```

SCRIPT CONTINUES: "**Paused** - Taking a break or switching to something else"

[Show code]:

```
// Pause when switching tasks
err := sessionMgr.Pause(sess.ID)

// Status: Active → Paused
fmt.Println(sess.Status) // "paused"
fmt.Println(sess.LastPausedAt) // Current time
```

SCRIPT CONTINUES: "**Resumed** - Back to work"

[Show code]:

```
// Resume work
err := sessionMgr.Resume(sess.ID)

// Status: Paused → Active
fmt.Println(sess.Status) // "active"
```

SCRIPT CONTINUES: "**Completed** - Successfully finished"

[Show code]:

```
// Mark as complete
err := sessionMgr.Complete(sess.ID)

// Status: Active → Completed
fmt.Println(sess.Status) // "completed"
fmt.Println(sess.EndedAt) // Current time
```

SCRIPT CONTINUES: "And if something goes wrong, you can mark it as **Failed** with a reason:"

[Show code]:

```
err := sessionMgr.Fail(sess.ID, "Tests failed, need to redesign")
// Status: Active → Failed
```

"This lifecycle tracking helps you understand where all your work stands."

[7:30-9:00] Creating and Managing Sessions

[ON SCREEN: Live coding demo]

SCRIPT: "Let's create a real session together. I'm going to implement user authentication."

[Demo - Type along]:

```
package main

import (
    "fmt"
    "dev.helix.code/internal/session"
)

func main() {
    // Create session manager
    mgr := session.NewManager()

    // Create auth session
    sess := mgr.Create(
        "implement-user-auth",
        session.ModeBuilding,
        "api-server",
    )

    // Add some tags for organization
```

```
    sess.AddTag("authentication")
    sess.AddTag("security")
    sess.AddTag("api")

    // Add metadata
    sess.SetMetadata("sprint", "23")
    sess.SetMetadata("assignee", "dev-team")

    // Start working
    mgr.Start(sess.ID)

    fmt.Printf("Started session: %s\n", sess.Name)
    fmt.Printf("Mode: %s\n", sess.Mode)
    fmt.Printf("Status: %s\n", sess.Status)
    fmt.Printf("Tags: %v\n", sess.Tags)

    // Do some work...
    // (This is where you'd use the AI, create conversations,
        etc.)

    // When done
    mgr.Complete(sess.ID)

    fmt.Printf("\nSession completed!\n")
    fmt.Printf("Duration: %v\n",
        sess.EndedAt.Sub(sess.StartedAt))
}
```

[Run the code]:

```
Started session: implement-user-auth
Mode: building
Status: active
Tags: [authentication security api]

Session completed!
Duration: 2h15m30s
```

SCRIPT CONTINUES: "See how easy that was? Create, start, work, complete. The session tracks everything - when you started, how long it took, all the context."

[9:00-10:00] Practical Tips and Conclusion

[ON SCREEN: Best practices list]

SCRIPT: “Before we wrap up, here are some practical tips:

1. Use descriptive names - ‘implement-user-auth’ is way better than ‘session-1’

2. Tag everything - Tags make finding sessions later super easy

3. One session at a time - Focus on one active session per area of work

4. Use the right mode - Don’t use Building mode when you’re debugging

5. Complete or fail sessions - Don’t leave orphaned sessions lying around

6. Store important context - Use SetContext() to save file positions, branch names, etc.

In the next video, we’ll explore advanced session features - queries, filtering, history management, and multi-session workflows.

Ready to level up? See you there!”

[ON SCREEN: “Next: Advanced Session Management”]

Code Examples

Complete Example

```go
package main

import (
    "fmt"
    "time"
    "dev.helix.code/internal/session"
)

func main() {
    mgr := session.NewManager()

    // Create session
    sess := mgr.Create(
        "implement-payment-gateway",
        session.ModeBuilding,
        "payment-service",
    )

    // Add organization
```

```
    sess.AddTag("payments")
    sess.AddTag("stripe")
    sess.SetMetadata("priority", "high")
    sess.SetContext("branch", "feature/payments")

    // Start
    if err := mgr.Start(sess.ID); err != nil {
        panic(err)
    }

    // Simulate work
    fmt.Println("Working on payment integration...")
    time.Sleep(2 * time.Second)

    // Pause for lunch
    mgr.Pause(sess.ID)
    fmt.Println("Paused for lunch")
    time.Sleep(1 * time.Second)

    // Resume
    mgr.Resume(sess.ID)
    fmt.Println("Back to work")
    time.Sleep(2 * time.Second)

    // Complete
    mgr.Complete(sess.ID)

    fmt.Printf("\n✓ Session completed!\n")
    fmt.Printf("  Total duration: %v\n",
        sess.EndedAt.Sub(sess.StartedAt))
}
```

Session Properties

```
// Access session properties
name := sess.Name
mode := sess.Mode
status := sess.Status
projectID := sess.ProjectID

// Metadata
sess.SetMetadata("version", "1.0.0")
version := sess.GetMetadata("version")
```

```
// Context (session-specific data)
sess.SetContext("current_file", "payment.go")
sess.SetContext("current_line", "42")

file := sess.GetContext("current_file")
line := sess.GetContext("current_line")

// Tags
sess.AddTag("feature")
hasTag := sess.HasTag("feature")
allTags := sess.Tags
```

Key Takeaways

1. Sessions organize work into focused periods with specific goals
2. Six modes optimize AI for different types of work
3. Five states track the session lifecycle
4. Tags and metadata enable powerful organization
5. Context preserves session-specific information
6. Simple API: Create → Start → Pause/Resume → Complete/Fail

Quiz Questions

1. What are the six session modes?
2. What is the lifecycle of a session from creation to completion?
3. How do you add tags to a session?
4. What's the difference between metadata and context?
5. When should you use Debugging mode vs Building mode?

Practice Exercise

Create a session for debugging a memory leak: 1. Use Debugging mode 2. Name it appropriately 3. Add relevant tags 4. Add metadata about the issue 5. Use context to track files being investigated 6. Complete when fixed

Solution: See `/examples/phase3/sessions/debugging-exercise.go`